```
        .duration(1000)
        .attr("transform", function(d) {
             return "translate("+ xScale(d.followers)
                         +","+yScale(d.following) +")";
         });

    d3.selectAll("line.link")
        .transition()
        .duration(1000)
        .attr("x1", function(d) {return xScale(d.source.followers);})
        .attr("y1", function(d) {return yScale(d.source.following);})
        .attr("x2", function(d) {return xScale(d.target.followers);})
        .attr("y2", function(d) {return yScale(d.target.following);});

    var xAxis = d3.svg.axis().scale(xScale).orient("bottom").tickSize(4);
    var yAxis = d3.svg.axis().scale(yScale).orient("right").tickSize(4);

    d3.select("svg").append("g").attr("transform",
             "translate(0,460)").call(xAxis);
    d3.select("svg").append("g").attr("transform",
             "translate(460,0)").call(yAxis);

    d3.selectAll("g.node").each(function(d){
      d.x = xScale(d.followers);
      d.px = xScale(d.followers);
      d.y = yScale(d.following);
      d.py = yScale(d.following);
    });

};
```

Notice that you need to update the `x` and `y` attributes of each node, but you also need to update the `px` and `py` attributes of each node. The `px` and `py` attributes are the previous x and y coordinates of the node before the last tick. If you don't update them, then the force layout thinks that the nodes have high velocity, and will violently move them from their new position.

If you didn't update the `x`, `y`, `px`, and `py` attributes, then the next time you started the force layout, the nodes would immediately return to their positions before you moved them. This way, when you restart the force layout with `force.start()`, the nodes and edges animate from their current position.

6.2.8 Optimization

The force layout is extremely resource-intensive. That's why it cools off and stops running by design. And if you have a large network running with the force layout, you can tax a user's computer until it becomes practically unusable. The first tip to optimization, then, is to limit the number of nodes in your network, as well as the number of edges. A general rule is no more than 100 nodes, unless you know your audience is going to be using the browsers that perform best with SVG, like Safari and Chrome.

But if you have to present more nodes and want to reduce the performance press, you can use `force.chargeDistance()` to set a maximum distance when computing the repulsive charge for each node. The lower this setting, the less structured the